

Scaling Data-Constrained Language Models

Niklas Muennighoff¹ Alexander M. Rush¹ Boaz Barak² Teven Le Scao¹
 Aleksandra Piktus¹ Nouamane Tazi¹ Sampo Pyysalo³ Thomas Wolf¹ Colin Raffel¹
¹ Hugging Face ² Harvard University ³ University of Turku
n.muennighoff@gmail.com

Abstract

The current trend of scaling language models involves increasing both parameter count and training dataset size. Extrapolating this trend suggests that training dataset size may soon be limited by the amount of text data available on the internet. Motivated by this limit, we investigate scaling language models in data-constrained regimes. Specifically, we run a large set of experiments varying the extent of data repetition and compute budget, ranging up to 900 billion training tokens and 9 billion parameter models. We find that with constrained data for a fixed compute budget, training with up to 4 epochs of repeated data yields negligible changes to loss compared to having unique data. However, with more repetition, the value of adding compute eventually decays to zero. We propose and empirically validate a scaling law for compute optimality that accounts for the decreasing value of repeated tokens and excess parameters. Finally, we experiment with approaches mitigating data scarcity, including augmenting the training dataset with code data or removing commonly used filters. Models and datasets from our 400 training runs are freely available at <https://github.com/huggingface/datablations>.

Data-Constrained Scaling Laws

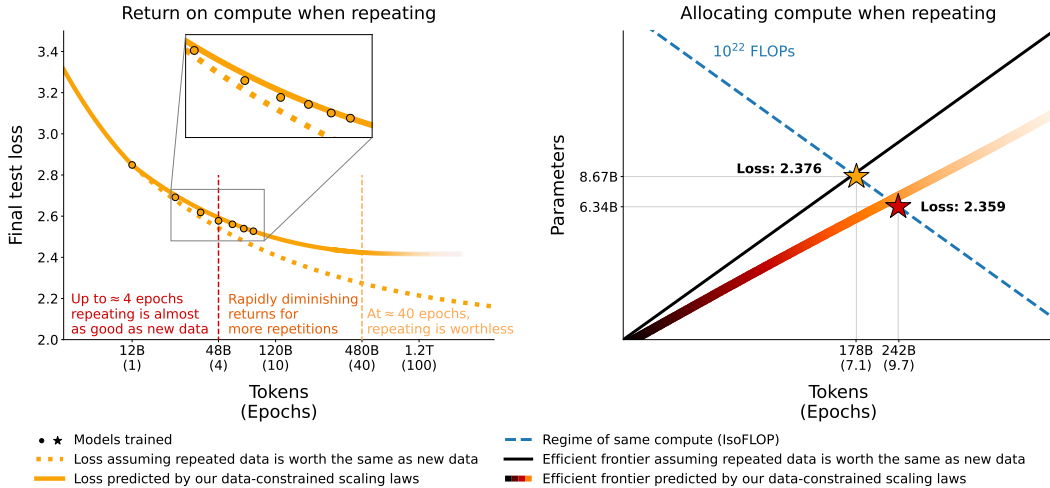


Figure 1: **Return and Allocation when repeating data.** (Left): Loss of LLMs (4.2B parameters) scaled on repeated data decays predictably (§6). (Right): To maximize performance when repeating, our data-constrained scaling laws and empirical data suggest training smaller models for more epochs in contrast to what assuming Chinchilla scaling laws [42] hold for repeated data would predict (§5).

1 Introduction

Recent work on compute-optimal language models [42] shows that many previously trained large language models (LLMs, which we define as having more than one billion parameters) could have attained better performance for a given compute budget by training a smaller model on more data. Notably, the 70-billion parameter Chinchilla model [42] outperforms the 280-billion parameter Gopher model [89] while using a similar compute budget by being trained on four times more data. Extrapolating these laws for compute allocation (hereafter "Chinchilla scaling laws") to a 530 billion parameter model, such as the under-trained MT-NLG model [99], would require training on a massive 11 trillion tokens, corresponding to more than 30 terabytes of text data. For most languages, available data is several orders of magnitude smaller, meaning that LLMs in those languages are already data-constrained. Villalobos et al. [112] estimate that even high-quality English language data will be exhausted by the year 2024 given the Chinchilla scaling laws and the trend of training ever-larger models. This motivates the question [112, 81]: what should we do when we run out of data?

In this work we investigate scaling large language models in a data-constrained regime, and whether training an LLM with multiple epochs of repeated data impacts scaling. Using multiple epochs is, of course, standard in machine learning generally; however, most prior large language models have been trained for a single epoch [51, 15] and some work explicitly advocates against reusing data [40]. An exception is the recent Galactica models [108] that were trained for 4.25 epochs and exhibit continually decreasing validation loss and improving downstream performance throughout training. However, the experiments of Galactica do not compare this setup to an alternative non-data-constrained model trained for one epoch on unique data. Without this comparison, it is difficult to quantify the trade-off between additional compute versus additional data collection.

Our main focus is to quantify the impact of multiple epochs in LLM training such that practitioners can decide how to allocate compute when scaling models. Toward this end, we assembled a battery of empirical training runs of varying data and compute constraints. Specifically, we train more than 400 models ranging from 10 million to 9 billion parameters for up to 1500 epochs and record final test loss. We use these results to fit a new *data-constrained scaling law* that generalizes the Chinchilla scaling law [42] to the repeated data regime and yields a better prediction of loss in this setting. Figure 1 summarizes our main results targeting the value of repeated data (*Return*) and optimal allocation of resources in that regime (*Allocation*). We find that, while models trained for a single epoch consistently have the best validation loss per compute, differences tend to be *insignificant* among models trained for up to 4 epochs and do not lead to differences in downstream task performance. Additional epochs continue to be beneficial, but returns eventually diminish to zero. We find that, in the data-constrained regime, allocating new compute to both more parameters and epochs is necessary, and that epochs should be scaled slightly faster. These findings suggest a simple way to continue scaling total training compute budgets further ahead in the future than the previously anticipated limits.

Finally, given the challenges imposed by data constraints, we consider methods complementary to repeating for improving downstream accuracy without adding new natural language data. Experiments consider incorporating code tokens and relaxing data filtering. For code, English LLMs, such as PaLM [19] or Gopher [89], are trained on a small amount of code data alongside natural language data, though no benchmarking was reported to justify that decision. We investigate training LLMs on a mix of language data and Python data at 10 different mixing rates and find that mixing in code is able to provide a $2\times$ increase in effective tokens even when evaluating only natural language tasks. For filtering, we revisit perplexity and deduplication filtering strategies on both noisy and clean datasets and find that data filtering is primarily effective for noisy datasets.

2 Background

Predicting the scaling behavior of large models is critical when deciding on training resources. Specifically, two questions are of interest: (*Allocation*) What is the optimal balance of resources? (*Return*) What is the expected value of additional resources? For scaling LLMs, the resource is compute (measured in FLOPs), and it can be allocated to training a larger model or training for more

steps.¹ The metric used to quantify progress is the model’s loss on held-out data, i.e. the ability to predict the underlying data as measured in the model’s cross-entropy [2, 42]. We aim to minimize the loss (L) subject to a compute resource constraint (C) via optimal allocation to N and D as:

$$\operatorname{argmin}_{N,D} L(N, D) \text{ s.t. } \text{FLOPs}(N, D) = C \quad (1)$$

Currently, there are established best practices for scaling LLMs. *Return* follows a power-law: loss scales as a power-law with the amount of compute used for training [39, 46, 6, 35, 7, 41]. *Allocation* is balanced: resources are divided roughly equally between scaling of parameters and data [42]. These scaling laws were established empirically by training LLMs and carefully extrapolating behavior.

Chinchilla [42] uses three methods for making scaling predictions:

- (*Fixed Parameters*) Train with a fixed model size but on varying amounts of data.
- (*Fixed FLOPs*) Train with fixed computation while parameters and training tokens vary.
- (*Parametric Fit*) Derive and fit a formula for the loss.

For the parametric fit, the loss (L) is a function of parameters (N) and training tokens (D):

$$L(N, D) = \frac{A}{N^\alpha} + \frac{B}{D^\beta} + E \quad (2)$$

Where $\{A, \alpha, B, \beta, E\}$ are learned variables fit using the training runs from the first two approaches [42]. Using these learned variables, they propose calculating the optimal allocation of compute (C) to N and D as follows:

$$\begin{aligned} N_{opt}(C) &= G(C/6)^a & D_{opt}(C) &= G^{-1}(C/6)^b \\ \text{where } G &= \left(\frac{\alpha A}{\beta B} \right)^{\frac{1}{\alpha+\beta}} & a &= \frac{\beta}{\alpha + \beta} & b &= \frac{\alpha}{\alpha + \beta} \end{aligned} \quad (3)$$

These methods lead to the conclusion that $\alpha \approx \beta$ and hence N and D should be scaled proportionally for compute-optimal training. As loss can be an imperfect proxy for performance on natural language tasks [123, 97, 105], they also validate their conclusions on various downstream tasks.

3 Method: Data-Constrained Scaling Laws

We are interested in scaling behavior in the data-constrained regime. Specifically, given a limited amount of unique data, what is the best *Allocation* of and *Return* for computational resources. Prior work [46, 42] assumes that the necessary data to support scaling is unlimited. Our aim is therefore to introduce a modified version of Equation 2 that accounts for data constraints and fit the terms in the modified scaling law to data from a large body of experiments.

The primary method we consider is *repeating* data, i.e. allocating FLOPs to multiple epochs on the same data. Given a budget of unique data D_C , we split the Chinchilla total data term D into two parts: the number of unique tokens used, U_D , and the number of repetitions, R_D (i.e. epochs - 1). Given total training tokens D and data budget D_C these terms are simply computed as $U_D = \min\{D_C, D\}$ and $R_D = (D/U_D) - 1$. When training for a single epoch like done in prior scaling studies, $R_D = 0$. We are thus interested in minimizing Equation 1 with the additional constraint of a data budget D_C :

$$\operatorname{argmin}_{N,D} L(N, D) \text{ s.t. } \text{FLOPs}(N, D) = C, U_D \leq D_C \quad (4)$$

Symmetrically, for mathematical convenience, we split the parameter term N into two parts: the base number of parameters needed to optimally fit the unique tokens U_N , and the number of times

¹In this work we use [46]’s approximation for the compute cost: $\text{FLOPs}(N, D) \approx 6ND$, where N denotes the number of model parameters and D denotes the number of tokens processed.